

Tarea 12

Diagrama de Clases

Andrés Serrano Modrego

```

classDiagram
    class CardType {
        ATTACK
        SKILL
        POWER
    }
    class Node {
        <<abstract>>
        +id: int
        +name: String
        +description: String
        +conditionType: String
        +targetValue: int
        +check_condition(progress: PlayerProgress): bool
    }
    class CombatManager {
        -player: Player
        -enemy: Enemy
        -turnNumber: int
        -energy: int
        +start_combat(player: Player, enemy: Enemy): void
        +player_turn(): void
        +enemy_turn(): void
        +end_turn(): void
        +check_end_conditions(): void
    }
    class GameManager {
        -currentRun: RunData
        -player: Player
        -dbManager: DatabaseManager
        +start_new_run(): void
        +load_run(): void
        +save_run(): void
        +return_to_menu(): void
    }
    class DatabaseManager {
        +get_all_cards(): Array<Card>
        +get_enemy_data(id: int): EnemyData
        +get_event_data(id: int): EventData
        +get_achievements(): Array<Achievement>
        +save_progress(progress: PlayerProgress): void
        +load_progress(): PlayerProgress
    }
    class RunData {
        -runId: int
        -seed: int
        -currentFloor: int
        -currentMapNode: int
        +reset(): void
    }
    class Player {
        -name: String
        -health: int
        -maxHealth: int
        -gold: int
        -deck: Deck
        -discardPile: Deck
        -relics: Array<Relic>
        +take_damage(amount: int): void
        +heal(amount: int): void
        +add_gold(amount: int): void
        +draw_cards(int): void
        +play_card(card: Card, target: Enemy): void
    }
    class Enemy {
        -id: int
        -name: String
        -health: int
        -maxHealth: int
        -baseDamage: int
        -intent: EnemyIntent
        +take_turn(player: Player): void
        +take_damage(amount: int): void
        +select_intent(): void
    }
    class EnemyData {
        -id: int
        -name: String
        -maxHealth: int
        -baseDamage: int
    }
    class EventData {
        -id: int
        -title: String
        -description: String
    }
    class Achievement {
        -id: int
        -name: String
        -description: String
        -conditionType: String
        -targetValue: int
        +check_condition(progress: PlayerProgress): bool
    }
    class PlayerProgress {
        -unlockedCards: Array<Card>
        -unlockedRelics: Array<Relic>
        -completedAchievements: Array<int>
        +unlock_card(cardId: int): void
        +unlock_relic(relicId: int): void
        +complete_achievement(achievementId: int): void
    }
    class Deck {
        -cards: Array<Card>
        +add_card(card: Card): void
        +remove_card(card: Card): void
        +shuffle(): void
        +draw(): Card
        +is_empty(): bool
    }
    class Relic {
        -id: int
        -name: String
        -description: String
        +apply_passive_effect(player: Player): void
    }
    class Shop {
        -availableCards: Array<Card>
        -prices: Map<Card,int>
        +buy(card: Card, player: Player): void
        +refresh(): void
    }
    class MapNode {
        -id: int
        -name: String
        -type: NodeType
        -floor: int
        -connections: Array<MapNode>
        +on_enter(player: Player): void
    }
    class Event {
        -id: int
        -title: String
        -description: String
        -options: Array<EventOption>
        +execute_option(index: int, player: Player): void
    }
    class EventOption {
        -text: String
        -result: EventResult
        +apply(player: Player): void
    }
    class EventResult {
        -goldChange: int
        -healthChange: int
        -cardReward: Card
        +apply(player: Player): void
    }
    class Card {
        -id: int
        -name: String
        -cost: int
        -description: String
        -type: CardType
        -rarity: Rarity
        -effect: Effect
        +can_play(currentEnergy: int): bool
        +apply_effect(player: Player, target: Enemy): void
    }
    class Effect {
        -amount: int
        -duration: int
        +apply(player: Player, target: Enemy): void
        +expire(player: Player, target: Enemy): void
    }
    class DamageEffect {
        +apply(player: Player, target: Enemy): void
    }
    class BlockEffect {
        +apply(player: Player, target: Enemy): void
    }
    class StatusEffect {
        -statusType: String
        +apply(player: Player, target: Enemy): void
    }

    CardType <|-- Card
    Node <|-- Achievement
    Node <|-- Event
    Node <|-- EventOption
    Node <|-- EventResult
    Node <|-- MapNode
    Node <|-- Relic
    Node <|-- Shop
    Node <|-- Card
    Node <|-- Effect
    Node <|-- DamageEffect
    Node <|-- BlockEffect
    Node <|-- StatusEffect

    CombatManager --> GameManager
    GameManager --> DatabaseManager
    GameManager --> RunData
    GameManager --> Player
    GameManager --> Enemy
    DatabaseManager --> Player
    DatabaseManager --> Enemy
    DatabaseManager --> Achievement
    DatabaseManager --> EventData
    DatabaseManager --> EventOption
    RunData --> Player
    RunData --> Enemy
    RunData --> Achievement
    RunData --> EventData
    RunData --> EventOption
    RunData --> EventResult
    RunData --> MapNode
    RunData --> Relic
    RunData --> Shop
    RunData --> Card
    RunData --> Effect
    RunData --> DamageEffect
    RunData --> BlockEffect
    RunData --> StatusEffect
    Player --> Deck
    Player --> DiscardPile
    Player --> Relics
    Player --> MapNode
    Player --> Event
    Player --> EventOption
    Player --> EventResult
    Player --> MapNode
    Player --> Relic
    Player --> Shop
    Player --> Card
    Player --> Effect
    Player --> DamageEffect
    Player --> BlockEffect
    Player --> StatusEffect
    Enemy --> MapNode
    Enemy --> Event
    Enemy --> EventOption
    Enemy --> EventResult
    Enemy --> MapNode
    Enemy --> Relic
    Enemy --> Shop
    Enemy --> Card
    Enemy --> Effect
    Enemy --> DamageEffect
    Enemy --> BlockEffect
    Enemy --> StatusEffect
    Achievement --> MapNode
    Achievement --> Event
    Achievement --> EventOption
    Achievement --> EventResult
    Achievement --> MapNode
    Achievement --> Relic
    Achievement --> Shop
    Achievement --> Card
    Achievement --> Effect
    Achievement --> DamageEffect
    Achievement --> BlockEffect
    Achievement --> StatusEffect
    EventData --> MapNode
    EventData --> Event
    EventData --> EventOption
    EventData --> EventResult
    EventData --> MapNode
    EventData --> Relic
    EventData --> Shop
    EventData --> Card
    EventData --> Effect
    EventData --> DamageEffect
    EventData --> BlockEffect
    EventData --> StatusEffect
    EventOption --> MapNode
    EventOption --> Event
    EventOption --> EventOption
    EventOption --> EventResult
    EventOption --> MapNode
    EventOption --> Relic
    EventOption --> Shop
    EventOption --> Card
    EventOption --> Effect
    EventOption --> DamageEffect
    EventOption --> BlockEffect
    EventOption --> StatusEffect
    EventResult --> MapNode
    EventResult --> Event
    EventResult --> EventOption
    EventResult --> EventResult
    EventResult --> MapNode
    EventResult --> Relic
    EventResult --> Shop
    EventResult --> Card
    EventResult --> Effect
    EventResult --> DamageEffect
    EventResult --> BlockEffect
    EventResult --> StatusEffect
    MapNode --> MapNode
    MapNode --> Event
    MapNode --> EventOption
    MapNode --> EventResult
    MapNode --> MapNode
    MapNode --> Relic
    MapNode --> Shop
    MapNode --> Card
    MapNode --> Effect
    MapNode --> DamageEffect
    MapNode --> BlockEffect
    MapNode --> StatusEffect
    Relic --> MapNode
    Relic --> Event
    Relic --> EventOption
    Relic --> EventResult
    Relic --> MapNode
    Relic --> Relic
    Relic --> Shop
    Relic --> Card
    Relic --> Effect
    Relic --> DamageEffect
    Relic --> BlockEffect
    Relic --> StatusEffect
    Shop --> MapNode
    Shop --> Event
    Shop --> EventOption
    Shop --> EventResult
    Shop --> MapNode
    Shop --> Relic
    Shop --> Shop
    Shop --> Card
    Shop --> Effect
    Shop --> DamageEffect
    Shop --> BlockEffect
    Shop --> StatusEffect
    Card --> MapNode
    Card --> Event
    Card --> EventOption
    Card --> EventResult
    Card --> MapNode
    Card --> Relic
    Card --> Shop
    Card --> Card
    Card --> Effect
    Card --> DamageEffect
    Card --> BlockEffect
    Card --> StatusEffect
    Effect --> MapNode
    Effect --> Event
    Effect --> EventOption
    Effect --> EventResult
    Effect --> MapNode
    Effect --> Relic
    Effect --> Shop
    Effect --> Card
    Effect --> Effect
    Effect --> DamageEffect
    Effect --> BlockEffect
    Effect --> StatusEffect
    DamageEffect --> MapNode
    DamageEffect --> Event
    DamageEffect --> EventOption
    DamageEffect --> EventResult
    DamageEffect --> MapNode
    DamageEffect --> Relic
    DamageEffect --> Shop
    DamageEffect --> Card
    DamageEffect --> Effect
    DamageEffect --> DamageEffect
    DamageEffect --> BlockEffect
    DamageEffect --> StatusEffect
    BlockEffect --> MapNode
    BlockEffect --> Event
    BlockEffect --> EventOption
    BlockEffect --> EventResult
    BlockEffect --> MapNode
    BlockEffect --> Relic
    BlockEffect --> Shop
    BlockEffect --> Card
    BlockEffect --> Effect
    BlockEffect --> DamageEffect
    BlockEffect --> BlockEffect
    BlockEffect --> StatusEffect
    StatusEffect --> MapNode
    StatusEffect --> Event
    StatusEffect --> EventOption
    StatusEffect --> EventResult
    StatusEffect --> MapNode
    StatusEffect --> Relic
    StatusEffect --> Shop
    StatusEffect --> Card
    StatusEffect --> Effect
    StatusEffect --> DamageEffect
    StatusEffect --> BlockEffect
    StatusEffect --> StatusEffect
  
```

The diagram illustrates the architecture of a card game system, organized into several interconnected components:

- Core Game Logic:**
 - GameManager**: Manages the overall game state, including the current run, player, and database manager.
 - CombatManager**: Handles the combat logic, including player and enemy turns, energy management, and end conditions.
- Data Management:**
 - DatabaseManager**: Manages the game's data, including cards, enemy data, event data, achievements, and progress.
 - RunData**: Stores data for the current game run, including the run ID, seed, current floor, and current map node.
- Game Entities:**
 - Player**: The main character, with attributes like name, health, max health, gold, deck, discard pile, and relics. It has methods for taking damage, healing, adding gold, drawing cards, and playing cards.
 - Enemy**: The opponent, with attributes like ID, name, health, max health, base damage, and intent. It has methods for taking turns, taking damage, and selecting intent.
 - EnemyData**: Stores data for enemies, including ID, name, max health, and base damage.
 - EventData**: Stores data for events, including ID, title, and description.
 - Achievement**: Represents game achievements, with attributes like ID, name, description, condition type, and target value. It has a method to check conditions.
 - PlayerProgress**: Tracks player progress, including unlocked cards, unlocked relics, and completed achievements. It has methods to unlock cards, unlock relics, and complete achievements.
 - Deck**: The player's deck of cards, with methods to add, remove, shuffle, draw, and check if empty.
 - Relic**: Special items that provide passive effects to the player.
 - Shop**: A place where players can buy cards, with methods to buy cards and refresh the shop.
 - MapNode**: Represents a location in the game world, with attributes like ID, name, type, floor, and connections to other nodes. It has a method to enter the node.
 - Event**: A game event, with attributes like ID, title, description, and options. It has a method to execute an option.
 - EventOption**: An option for an event, with attributes like text and result. It has a method to apply the option.
 - EventResult**: The result of an event, including gold change, health change, and card reward. It has a method to apply the result.
- Card System:**
 - Card**: The basic unit of the game, with attributes like ID, name, cost, description, type, rarity, and effect. It has methods to check if it can be played and to apply its effect.
 - Effect**: A game effect, with attributes like amount and duration. It has methods to apply and expire the effect.
 - DamageEffect**: A specific type of effect that deals damage.
 - BlockEffect**: A specific type of effect that blocks an action.
 - StatusEffect**: A specific type of effect that changes a status.